

Ansible Deployment for Mac and Windows Guides

- 1. Mac Local Deployment {#mac-local}
 - Prerequisites
 - Inventory Configuration
 - Deployment Structure
 - Deployment Steps
- 2. Mac Containerized Deployment {#mac-containerized}
 - Prerequisites
 - Inventory Configuration
 - Deployment Structure
 - Deployment Steps
- 3. Windows Local Deployment {#windows-local}
 - Prerequisites
 - Inventory Configuration
 - Deployment Structure
 - Deployment Steps
- 4. Windows Containerized Deployment {#windows-containerized}
 - Prerequisites
 - Inventory Configuration
 - Deployment Structure
 - Deployment Steps
 - Common Operations for All Deployments
 - Pre-deployment Checks
 - Troubleshooting
 - Mac Issues
 - Windows Issues
 - Security Best Practices
 - Multi-Node Configuration

1. Mac Local Deployment {#mac-local}

Prerequisites

- macOS operating system
- Homebrew package manager
 - *Ansible: brew install ansible*
- Node.js and npm

Inventory Configuration

```
# ansible-mac-local/inventory/inventory.yml
[nodes]
localhost ansible_connection=local
```

Deployment Structure

```
# main.yml
- name: Install Nginx on macOS
  hosts: localhost
  gather_facts: yes
  tasks:
    - name: Install Nginx
      community.general.homebrew:
        name: nginx
        state: present

    - name: Verify Nginx installation
      ansible.builtin.command: which nginx
      register: nginx_check

    - name: Check if http-server is installed
      ansible.builtin.command: which http-server
      register: http_server_check
      ignore_errors: true

    - name: Install http-server globally using npm
      ansible.builtin.command: npm install -g http-server
      when: http_server_check.rc != 0

    - name: Start http-server in dist folder
      ansible.builtin.command: http-server -p 4200
      args:
        chdir: /Users/path/to/dist/folder
        async: 60
        poll: 0
```

Deployment Steps

```
ansible-playbook -i ./ansible-mac-local/inventory/inventory.yml ./ansible-mac-local/playbooks/main.yml
```

2. Mac Containerized Deployment {#mac-containerized}

Prerequisites

- Docker Desktop for Mac
- Ansible:

```
brew install ansible
```

Inventory Configuration

```
# ansible-mac-containerized/inventory/inventory.ini
[nodes]
node1 ansible_host=<node_ip> ansible_user=<username> ansible_password=<password>
```

Deployment Structure

```
# main.yml
---
- name: Install Docker
  ansible.builtin.import_playbook: install-docker.yml

- name: Initialize Docker Swarm
  ansible.builtin.import_playbook: install-docker-swarm.yml

- name: Deploy Spa
  ansible.builtin.import_playbook: install-spa.yml

- name: Deploy Visualizer
  ansible.builtin.import_playbook: install-docker-visualizer.yml
```

Deployment Steps

```
ansible-playbook -i ./ansible-mac-containerized/inventory/inventory.ini ./ansible-mac-containerized/playbooks/main.yml
```

3. Windows Local Deployment {#windows-local}

Prerequisites

- .NET Runtime
- OpenSSH Server
- PowerShell

Inventory Configuration

```
# ansible-windows-local/inventory/inventory.ini
[windows]
win-server ansible_host=192.168.101.181

[windows:vars]
ansible_user=user
ansible_password=1234
ansible_connection=ssh
ansible_shell_type=powershell
ansible_ssh_common_args='-o StrictHostKeyChecking=no'
```

Deployment Structure

```

# main.yml
- name: Run dotnet application on Windows
  hosts: windows
  gather_facts: no
  tasks:
    # Gateway
    - name: Execute Gateway
      win_shell: |
        Start-Process 'dotnet' -Wait -ArgumentList 'C:\Users\user\Documents\FTP\arc-zip\windoes-arc-publish\publish-auth\publish-auth.dll'
      register: dotnet_run_result
      ignore_errors: yes

    # Authentication
    - name: Execute Authentication API
      win_shell: |
        Start-Process 'dotnet' -Wait -ArgumentList 'C:\Users\user\Documents\FTP\arc-zip\windoes-arc-publish\publish-auth\Authentication.API.dll'
      register: dotnet_run_result
      ignore_errors: yes

    # Configuration
    - name: Execute Configuration API
      win_shell: |
        Start-Process 'dotnet' -Wait -ArgumentList 'C:\Users\user\Documents\FTP\arc-zip\windoes-arc-publish\publish-confi\Configuration.API.dll'
      register: dotnet_run_result
      ignore_errors: yes

    # VPM
    - name: Execute VPM
      win_shell: |
        Start-Process 'dotnet' -Wait -ArgumentList 'C:\Users\user\Documents\FTP\arc-zip\windoes-arc-publish\publish-vpm\VPM.dll'
      register: dotnet_run_result
      ignore_errors: yes

```

Deployment Steps

```

ansible-playbook -i ./ansible-windows-local/inventory/inventory.ini ./ansible-windows-local/playbooks/main.yml

```

4. Windows Containerized Deployment {#windows-containerized}

Prerequisites

- Docker for Windows
- OpenSSH Server
- PowerShell

Inventory Configuration

```

# ansible-windows-containerized/inventory/inventory.ini
[windows]
win-server ansible_host=192.168.101.181

[windows:vars]
ansible_user=user
ansible_password=1234
ansible_connection=ssh
ansible_shell_type=powershell
ansible_ssh_common_args='-o StrictHostKeyChecking=no'

```

Deployment Structure

```
# main.yml
---
- name: Install Docker
  ansible.builtin.import_playbook: install-docker.yml

- name: Initialize Docker-Swarm
  ansible.builtin.import_playbook: install-docker-swarm.yml

- name: Load Docker image
  ansible.builtin.import_playbook: load-docker-images.yml

# Infrastructure Services
- name: Deploy Visualizer
  ansible.builtin.import_playbook: install-docker-visualizer.yml

- name: Deploy Postgres
  ansible.builtin.import_playbook: ./docker-stack/create-postgres.yml

- name: Deploy Keycloak
  ansible.builtin.import_playbook: ./docker-stack/create-keycloak.yml

- name: Deploy Mediamtx
  ansible.builtin.import_playbook: ./docker-stack/create-mediatrix.yml

# Microservices
- name: Deploy Authentication Webapi
  ansible.builtin.import_playbook: ./microservices/create-authentication.yml

- name: Deploy Configuration
  ansible.builtin.import_playbook: ./microservices/create-configuration.yml

- name: Deploy vpm
  ansible.builtin.import_playbook: ./microservices/create-vpm.yml

- name: Deploy gateway
  ansible.builtin.import_playbook: ./microservices/create-gateway.yml

- name: Deploy Arc-WebSpa
  ansible.builtin.import_playbook: ./microservices/create-arc-spa.yml
```

Deployment Steps

```
ansible-playbook -i ./ansible-windows-containerized/inventory/inventory.ini ./ansible-windows-containerized/playbooks/main.yml
```

Common Operations for All Deployments

Pre-deployment Checks

1. Verify network connectivity
2. Check required services
3. Validate inventory files
4. Test authentication

Troubleshooting

Mac Issues

```
# Check Docker status
docker info
docker stack ls

# Check Nginx status
brew services status nginx
nginx -t

# Check logs
tail -f /usr/local/var/log/nginx/error.log
```

Windows Issues

```
# Check SSH service
Get-Service sshd

# Check Docker service
Get-Service docker

# Check running .NET processes
Get-Process | Where-Object {$_.ProcessName -like "*dotnet*"}
```

Security Best Practices

1. Use SSH keys instead of passwords
2. Implement proper key management

Multi-Node Configuration

```
[nodes]
node1 ansible_host=192.168.1.101
node2 ansible_host=192.168.1.102

[windows]
win1 ansible_host=192.168.1.201
win2 ansible_host=192.168.1.202
```

Here's the link to reference the GitLab code:

https://gitlab.com/sharan.phadke/arc/-/tree/ansible-mac-and-windows-deployment?ref_type=heads